Travaux Dirigés 2 : NumPy Avancé

ali.zainoul.az@gmail.com

23 mars 2026

Introduction

Ce TD approfondit l'utilisation de NumPy, en se concentrant sur des concepts avancés comme le *broadcasting*, l'algèbre linéaire, la génération aléatoire, le filtrage avancé, les tableaux structurés, et la performance. Chaque exercice est conçu pour renforcer votre compréhension théorique et pratique, avec des exemples concrets et des conseils pour optimiser votre code.

Aide-mémoire NumPy Avancé

Broadcasting

Opération	Description
`A + b`	Ajoute le vecteur `b` à chaque ligne de la matrice `A` (si `A` est 2D et `b` 1D).
`A * b`	Multiplie chaque ligne de `A` par le vecteur `b` (broadcasting compatible).
`A + B[:, np.newaxis]`	Ajoute une dimension à `B` pour le broadcasting.
`np.newaxis`	Utilisé pour ajouter une dimension de taille 1.

Algèbre linéaire

Fonction	Usage
`np.dot(A, B)`	Produit matriciel de `A` et `B`.
`A @ B`	Équivalent à `np.dot(A, B)` (Python 3.5+).
`np.linalg.det(A)`	Calcule le déterminant de la matrice carrée `A`.
`np.linalg.inv(A)`	Calcule l'inverse de la matrice carrée `A`.
`np.linalg.eig(A)`	Calcule les valeurs et vecteurs propres de `A`.
`np.linalg.svd(A)`	Décomposition en valeurs singulières de `A`.

Génération aléatoire

Fonction	Usage
`np.random.normal(mu, sigma, size)`	Génère des échantillons d'une loi normale.
`np.random.uniform(low, high, size)`	Génère des échantillons d'une loi uniforme.
`np.random.exponential(scale, size)`	Génère des échantillons d'une loi exponentielle.
`np.random.shuffle(arr)`	Mélange les éléments de `arr` sur place.
`np.random.choice(arr, size, replace)`	Tire des échantillons aléatoires de `arr`.
`np.random.seed(seed)`	Fixe la graine pour la reproductibilité.

Filtrage avancé

Fonction	Usage
`np.where(condition, x, y)`	Remplace les éléments selon une condition.
`arr[list_of_indices]`	*Fancy indexing* pour accéder à des éléments spécifiques.
`np.isnan(arr)`	Crée un masque pour les valeurs NaN.
`np.isfinite(arr)`	Crée un masque pour les valeurs finies.

Tableaux structurés

Fonction	Usage
`np.dtype([('name', type), ...])`	Définit un type de données structuré.
`arr['field']`	Accède au champ `field` du tableau structuré `arr`.
`np.sort(arr, order='field')`	Trie le tableau structuré selon le champ `field`.

Sauvegarde et chargement

Fonction	**Usage**
`np.save('file.npy', arr)`	Sauvegarde le tableau `arr` dans un fichier binaire.
`np.load('file.npy')`	Charge un tableau depuis un fichier .npy.
`np.savez('file.npz', a=arr1, b=arr2)`	Sauvegarde plusieurs tableaux dans un fichier .npz.
`np.genfromtxt('file.csv', delimiter=',')`	Charge des données depuis un fichier CSV.

1 Exercice 1 : Broadcasting avancé

Objectif : Maîtriser le mécanisme de *broadcasting* pour effectuer des opérations sur des tableaux de dimensions différentes sans utiliser de boucles explicites.

1.1 Comprendre le broadcasting

- **Créer une matrice 3×4 et un vecteur de taille 4** : La matrice représente des données multidimensionnelles, et le vecteur sera ajouté à chaque ligne.
- **Ajouter le vecteur à chaque ligne** : NumPy étend le vecteur pour qu'il corresponde à la forme de la matrice.
- **Multiplier chaque ligne par un coefficient différent** : Utilisez un vecteur de coefficients de même longueur que le nombre de lignes.

Exemple : Si `matrix` est de forme (3, 4) et `vector` de forme (4,), alors `matrix + vector` ajoute `vector` à chaque ligne de `matrix`.

Conseil : Le *broadcasting* suit des règles strictes : les dimensions doivent être compatibles (soit égales, soit l'une d'elles vaut 1).

1.2 Broadcasting en 3D

- **Créer un tableau 3D** : Par exemple, de forme (2, 3, 4).
- **Ajouter une matrice 2D** : La matrice doit être de forme compatible (par exemple, (3, 4)).
- **Multiplier par un vecteur** : Le vecteur doit être de taille compatible avec l'axe concerné.
- **Normaliser chaque tranche** : Utilisez `axis=(1, 2)` pour calculer la moyenne et l'écart-type le long des deux derniers axes.

Remarque : La normalisation consiste à soustraire la moyenne et diviser par l'écart-type pour chaque tranche 2D du tableau 3D.

2 Exercice 2 : Algèbre linéaire

Objectif : Découvrir les fonctionnalités d'algèbre linéaire de NumPy, essentielles pour le traitement de données multidimensionnelles.

2.1 Produits matriciels

- **Créer deux matrices compatibles** : La première matrice doit avoir un nombre de colonnes égal au nombre de lignes de la deuxième matrice.
- **Calculer le produit matriciel** : Utilisez l'opérateur `@` ou la fonction `np.dot`.
- **Étudier les dimensions** : Le produit de deux matrices de formes (m, n) et (n, p) donne une matrice de forme (m, p).

Exemple : Si `A` est de forme (3, 4) et `B` de forme (4, 2), alors `A @ B` donne une matrice de forme (3, 2).

2.2 Déterminant et inverse

- **Calculer le déterminant** : Utilisez `np.linalg.det`. Le déterminant est un scalaire qui caractérise la matrice carrée.
- **Calculer l'inverse** : Utilisez `np.linalg.inv`. L'inverse d'une matrice `A` est une matrice `B` telle que `A @ B = I`.
- **Vérifier le résultat** : Multipliez la matrice par son inverse et vérifiez que le résultat est proche de la matrice identité.

Remarque : Seules les matrices carrées et inversibles (déterminant non nul) ont un inverse.

2.3 Valeurs propres

- **Calculer valeurs et vecteurs propres** : Utilisez `np.linalg.eig`. Les valeurs propres sont des scalaires qui caractérisent la transformation linéaire représentée par la matrice.
- **Vérifier la relation fondamentale** : Pour une matrice `A` et un vecteur propre `v` associé à la valeur propre λ, on a `A @ v =` λ `* v`.

Conseil : Les valeurs propres sont utiles pour comprendre la stabilité et la dynamique des systèmes linéaires.

3 Exercice 3 : Génération aléatoire

Objectif : Maîtriser la génération de nombres aléatoires selon différentes distributions, utile pour la simulation et les tests.

3.1 Distributions

- **Générer des lois normale, uniforme et exponentielle** :
  - Loi normale : `np.random.normal(moyenne, écart-type, taille)`.
  - Loi uniforme : `np.random.uniform(min, max, taille)`.
  - Loi exponentielle : `np.random.exponential(échelle, taille)`.
- **Comparer aux valeurs théoriques** : Calculez la moyenne et l'écart-type des échantillons générés et comparez-les aux paramètres théoriques.

Exemple : `np.random.normal(0, 1, 1000)` génère 1000 échantillons d'une loi normale centrée réduite.

3.2 Échantillonnage

- **Mélanger un tableau** : Utilisez `np.random.shuffle` pour mélanger les éléments d'un tableau *sur place*.
- **Tirer des échantillons** : Utilisez `np.random.choice` pour tirer des échantillons avec ou sans remplacement.

Conseil : Pour reproduire des résultats aléatoires, fixez la graine avec `np.random.seed`.

4 Exercice 4 : Filtrage avancé

Objectif : Utiliser des techniques avancées de filtrage pour nettoyer, transformer et analyser des données.

4.1 Conditions avec where

- **Nettoyer des données** : Remplacez les valeurs négatives par zéro avec `np.where(condition, x, y)`.
- **Catégoriser** : Créez des catégories en fonction de conditions (par exemple, "faible", "moyen", "élevé").

Exemple : `np.where(values < 0, 0, values)` remplace les valeurs négatives de `values` par 0.

4.2 Fancy indexing

- **Extraire des éléments** : Utilisez des listes d'indices pour extraire des éléments spécifiques.
- **Modifier des valeurs** : Modifiez des éléments spécifiques en utilisant des listes d'indices.

Remarque : Le *fancy indexing* permet d'accéder à des éléments non contigus et de les modifier efficacement.

5 Exercice 5 : Tableaux structurés

Objectif : Découvrir les tableaux structurés, qui permettent de stocker des données hétérogènes (comme des bases de données).

- **Créer un dataset structuré** : Définissez un type de données structuré avec `np.dtype` et créez un tableau de ce type.
- **Trier, filtrer et analyser** : Utilisez les noms de champs pour accéder aux données et effectuer des opérations.

Exemple : Un tableau structuré peut contenir des champs comme "nom", "âge", et "note", chacun avec un type spécifique.

6 Exercice 6 : Sauvegarde et chargement

Objectif : Apprendre à sauvegarder et charger des tableaux NumPy, ainsi qu'à manipuler des fichiers CSV.

- **Sauvegarder en .npy et .npz** :
  - `np.save` pour sauvegarder un seul tableau.
  - `np.savez` pour sauvegarder plusieurs tableaux dans un seul fichier.
- **Charger des données** : Utilisez `np.load` pour charger des fichiers .npy ou .npz.
- **Manipuler des fichiers CSV** : Utilisez `np.genfromtxt` pour lire des données depuis un fichier CSV.

Conseil : Les fichiers .npy et .npz sont optimisés pour les tableaux NumPy et préservent les métadonnées comme le type et la forme.

7 Exercice 7 : Performance

Objectif : Comparer les performances des boucles Python et des opérations vectorisées, et comprendre la différence entre vues et copies.

- **Comparer boucles vs vectorisation** : Mesurez le temps d'exécution d'une opération effectuée avec une boucle et avec des opérations vectorisées.
- **Comprendre vues vs copies** :
  - Une *vue* (*view*) partage les données avec le tableau original.
  - Une *copie* (*copy*) crée un nouveau tableau indépendant.

Exemple : `arr[::2]` crée une vue, tandis que `arr.copy()` crée une copie.

8 Exercice 8 : Projet - Monte Carlo

Objectif : Estimer la valeur de π en utilisant une simulation de Monte Carlo, une méthode probabiliste puissante pour approximer des résultats numériques.

- **Estimer π par simulation** :
  - Générez des points aléatoires dans un carré de côté 2 centré à l'origine.
  - Comptez le nombre de points qui tombent dans le cercle de rayon 1 centré à l'origine.
  - Estimez π en utilisant la proportion de points dans le cercle.
- **Analyser la précision** : Comparez votre estimation à la valeur théorique de π et analysez l'erreur en fonction du nombre d'échantillons.

Formule : Si n est le nombre total de points et m le nombre de points dans le cercle, alors $\pi \approx 4 \times \frac{m}{n}$.

Résumé des concepts clés

Broadcasting

- Le *broadcasting* permet d'effectuer des opérations entre tableaux de formes différentes.
- Les dimensions doivent être compatibles (égales ou de taille 1).
- Utilisez `np.newaxis` pour ajouter des dimensions.

Algèbre linéaire

- `np.dot` et `@` pour le produit matriciel.
- `np.linalg.det`, `np.linalg.inv`, `np.linalg.eig` pour les opérations matricielles.
- Les valeurs propres et vecteurs propres caractérisent les transformations linéaires.

Génération aléatoire

- `np.random.normal`, `np.random.uniform`, `np.random.exponential` pour générer des échantillons.
- `np.random.seed` pour la reproductibilité.
- `np.random.shuffle` et `np.random.choice` pour l'échantillonnage.

Filtrage avancé

- `np.where` pour le remplacement conditionnel.
- *Fancy indexing* pour accéder/modifier des éléments spécifiques.
- Les masques booléens sont puissants pour le filtrage.

Tableaux structurés

- Permettent de stocker des données hétérogènes.
- Utilisez `np.dtype` pour définir la structure.
- Accès aux champs par nom : `arr['field']`.

Sauvegarde et chargement

- `np.save` et `np.load` pour les fichiers binaires.
- `np.savez` pour sauvegarder plusieurs tableaux.
- `np.genfromtxt` pour les fichiers CSV.

Performance

- Privilégiez les opérations vectorisées aux boucles.
- Les vues partagent les données, les copies créent de nouveaux tableaux.
- Mesurez le temps d'exécution avec `%timeit` (IPython).